

46-926, Fall 2020: Homework #4

Due **10:45am**, Wednesday December 2, 2020.

Homework should be submitted electronically on canvas before the deadline. Please submit your homework as a single file (pdf preferred, canvas struggles with html). We recommend using jupyter notebooks to prepare your homework.

Do all computational problems in python unless otherwise noted. Please post any questions on the Piazza discussion board.

Update 11/24/2020 at 1:45pm: Removed problem 4c from the homework, since we have not covered the necessary material.

Comments on this 2 week assignment: Problem 1 and 2 are problems that you should be able to do right now. Problem 2 will rely on some code that was in our notebook in class, but that we didn't get a chance to talk about much. It should be pretty self-explanatory, but do come talk on Piazza if there are any gaps. Problem 3 and 4 will build on material that we'll talk about on Monday, November 23 (since it's a 2 week homework).

Problems 1 and 3 are the longer ones, 2 and 4 are shorter. We were right on the verge last class of having all the information that you need for Problem 3. I've posted a notebook on additive models that is very relevant to Problem 3 if you want to get a head start. It should be pretty feasible, but again, you'll still have have 9 days from Monday until the deadline if you want to wait until we technically cover the details.

Problem 4, as written, is difficult to do without the lecture on Nov 23. I also reserve the right to make modifications to the end of the assignment (changing or substituting Problem 4, or the possible but unlikely addition of a Problem 5) after we see how that lecture goes.

1. *Understanding how splines are fit.* This problem is intended to walk you through a subset of the claims I made in class. Consider a smoothing spline fit to n points, $(x_1, y_1), \dots, (x_n, y_n)$. For a fixed λ , we solve

$$\min_f \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int (f''(x))^2 dx$$

Suppose that you know:

- The solution to such a problem must be a natural cubic spline with knots at the x_i .
- Natural cubic splines can be represented in terms of a basis of known functions of x :

$$f(x) = \beta_0 + \sum_{k=1}^K \beta_k b_k(x)$$

Here the $b_k(x)$ are the known basis functions. (Note, I'm just using K instead of $K + 3$ in the notes, just to keep things clean. Just imagine you redefined K , it's fine.)

This means that to fit your f , you only need to figure out the β_k .

- Since we know each of the $b_k(x)$ functions, we could evaluate them at all of our training coordinates x_i . Show that you can rewrite the first part of your optimization objective, $\sum_{i=1}^n (y_i - f(x_i))^2$ entirely in terms of the y_i , $b_k(x_i)$, and β_k . Write the term entirely in matrix/vector notation, as we do in regression. You will need to define a matrix of your $b_k(x_i)$ values.
- Similarly, rewrite your penalty term, $\int (f''(x))^2 dx$ in terms of your functions $b_k(x)$ (and/or their derivatives), β_k . Convert this to a matrix/vector expression. *Hint:* You can assume you know all the derivatives of your $b_k(x)$. Your matrices are allowed to have elements that depend on integrals of your $b_k(x)$ functions (and their derivatives). It may help to first think about what $f''(x)$ looks like in terms of the $b_k(x)$ functions, and then move along to squaring and integrating it.
- Combining these two pieces, you should now have an objective that is written entirely in terms of matrices and vectors. The only unknowns will be your vector of β coefficients. If everything has gone right, your objective will now remind you very strongly of ridge regression.

Thinking back to your homework from last week, find a solution closed form solution for β (in terms of the matrices you defined above).

(I'm not looking for a proof or derivation here. An argument by intuition and analogy is fine.)

The result from this problem means that you can calculate your coefficient β as a linear operator on your vector of Y . Once you have this β , your spline would just be $\beta_0 + \sum_{k=1}^K \beta_k b_k(x)$.

2. *You may have seen this data in Data Science as an example of kernel regression, depends on the year, but we will use it again to look at smoothing splines.*

One approach to estimating the forward rate function (which can be used to calculate bond prices and yield curves) is to obtain many noisy point estimates of the forward rate at different times by comparing pairs of zero-coupon bonds, and then smooth them to get a more stable estimate of the curve.

- (a) Load `forward_rates.csv`. This contains a set of empirical forward rate estimates for a range of times (courtesy of Ruppert and Matteson, 2015, which you can also read – free on SpringerLink – for a better description of the data). Plot the empirical rates against time.

You should see that the points make a very noisy estimate of a smooth curve. It should be clear that a linear function, or even a quadratic one, would not capture this shape very well.

- (b) Fit a smoothing spline to this data. In python, use `LinearGAM` like we did in class. In R, use `smooth.spline`. (Up to you.) Cross-validate for a reasonable λ . Plot your fit on the same plot as the points. Can you manage to get a believable, smooth fit?

- (c) Computing the yield curve from this forward rate function involves definite integrals of your estimated smooth function. Splines are particularly nice for doing this. Think about the nature of cubic splines, and explain briefly why they can be integrated quickly and in essentially closed form (no numeric integration).

To think about: Compare this to nonparametric estimates from kernel regression, where you would need to evaluate the function on a fine grid of points and then carry out numerical integration.

3. This problem explores additive models for prediction. You will consider a bike rental data set, and forecast utilization based on time and weather. This information could be useful for tasks like inventory management and maintenance; we will explore this impact in terms of loss later in the problem.

Download `bike_train.csv` and `bike_test.csv` from canvas. These have daily data for two years. Load the data and get familiar with it.

- `dayofyear`, `dayofweek` are the 0-indexed day of the year and of the week. I've pretended that 2012 is not a leap year.

- **workday** is a binary indicator of whether it is a workday (1) or a weekend/holiday (0).
- **weathertype** is a four level variable for type of weather, with higher numbers indicating worse weather: 1: Clear, Few clouds, Partly cloudy, Partly cloudy; 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist; 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds; 4: Heavy Rain + Ice Pellets + Thunderstorm + Mist, Snow + Fog
- **temp**, **humidity**, **windspeed** are all what they sound like.
- **rentals** is the number of rentals, your target variable for prediction.

Important: We will be fitting on data from 2011 and testing on data from 2012. However, this service became much more popular in 2012. To give your methods a fighting chance, you get the additional information that your overall ridership is up about 1.6 times. That is, *whenever you make a prediction on the test data, multiply your prediction by 1.6.*

Note: You will be fitting a few different models. Store each of them as you go, since we will evaluate them in different ways later in the problem.

- Suppose that you want to predict the number of rentals (**rentals**) based on **dayofyear**, **dayofweek**, **workday**, **weathertype**, **temp**, **humidity**, **windspeed**. Fit a *linear model* to make this prediction.
Make predictions on your test data. Remember to scale your guesses by 1.6 to account for overall growth. What is your MSE on the test data?
- Now fit an additive model. Some of your variables should definitely be smooth, continuous terms, like **dayofyear**, **temp**, **humidity**, **windspeed**. For others, you may want to use factors if smoothness doesn't make sense.
Make predictions on your test data (again scaling by 1.6). What is your MSE now?
- Plot the functions for each of your smooth covariates. What nonlinear trends do you see that a linear model would miss? Give a very brief interpretation for each of your smooth fits.
- It turns out that MSE is a strange measure for our inventory management. Consider the following more realistic loss function (though also idealized): For every unit that we are understocked, we lose \$5 because we miss out on a rental. For every unit that we overstock, we lose \$1 because we maintain a surplus

inventory:

$$\mathcal{L}(Y, \hat{Y}) = \begin{cases} 5(Y - \hat{Y}) & \text{if } Y > \hat{Y} \\ \hat{Y} - Y & \text{if } Y < \hat{Y} \end{cases}$$

The following function computes this new loss:

```
def real_loss(true, guess):  
    return(np.mean(np.where(true>guess, 5*(true-guess), guess-true)))
```

What is this loss for your linear and your spline model?

- (e) As you saw on your first homework, when the loss is asymmetric, the mean is not a great predictor. One solution would be to optimize the correct loss (in this case, the corresponding method is called *quantile regression*, and actually corresponds to the weird loss function from quiz 1!). Instead, let's consider adjusting our estimates with a hack.

Since it hurts more to undershoot than overshoot, we want to skew our estimates upward to improve our loss. It turns out that a reasonable adjustment would be to add the 83.333% (5/6) quantile of our residual distribution to our estimates. (We will discuss this further in class.) Because of the weird scaling across years and to simplify this homework, we'll cheat and use the test data to calculate this quantile just as an illustration. Other approaches are possible.

Using the `test` data, calculate the 5/6th quantile of the residuals $Y - \hat{Y}$ for each predictor (where you have scaled $\hat{Y}$ by 1.6 already). In python, you can use `np.percentile`; in R, you can use `quantile`. Now build new predictors for both your linear method and your additive model by adding the corresponding quantile to your predictions. That is:

$$\hat{Y}_{new} = \hat{Y} + \text{quantile}$$

Evaluate these new predictions on the test set in terms of the asymmetric loss function. How does the performance compare to the unadjusted estimator from the previous part? How does the additive model compare to the linear model now?

What would be your final choice of estimator?

4. *Best classification when your losses are asymmetric.* Consider a two-class ($\{0, 1\}$) classification problem. We will see in class that for 0-1 loss, the optimal classification rule is

$$f(x) = \begin{cases} 1 & \text{if } P(Y = 1 \mid X = x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

if we assume that we know everything about the distribution of (X, Y) .

Suppose instead that our loss is

	True $Y = 1$	True $Y = 0$
Guess $Y = 1$	0	L_{01}
Guess $Y = 0$	L_{10}	0

- (a) Again assuming that we know everything about the distribution, find the best possible classification function $f(x)$ for minimizing this new, more general loss. The calculation will be very similar to the one in class.
- (b) If $L_{10} > L_{01}$ (so that false negatives are worse than false positives), how does this classification rule differ from the 0-1 loss rule we derived in class? Why does this make sense? (You can just answer with two short sentences.)

To think about: How should your classifiers change if your class frequencies are different between your training and test data?